GridAid

Resilience-Aware Emergency Resource Planning — 2026 Congressional App Challenge Project Dossier

Working Competition Dossier

September 2026

Project thesis

GridAid is a browser-based resilience planning platform that combines road-network algorithms, facility-location optimization, machine learning, Monte Carlo simulation, and disruption analysis to identify limited emergency-resource configurations that remain effective when conditions change.

The project is built around a planning problem rather than a dispatch problem. A community may know that it has ten emergency kits, several temporary response teams, a set of public facilities, or a limited number of other critical resources. The obvious question is where those resources should be placed. The more difficult question is whether that placement still performs well when a road closes, winter weather slows travel, one resource becomes unavailable, or demand temporarily shifts.

GridAid models those conditions before an incident occurs.

The competition prototype is a **planning and simulation tool**, not a 911 dispatch system, medical application, or promise about real emergency outcomes. It uses public, synthetic, and documented data; exposes the assumptions behind its results; and validates its algorithms independently.

The project has five technical pillars:

1. **Geospatial computing:** build a real road graph and spatial demand layers.
2. **Optimization:** solve static and resilience-aware resource-placement problems.
3. **Machine learning:** produce an Iowa transportation incident-risk demand layer from historical public data.
4. **Simulation:** stress-test configurations across thousands of modeled incidents and disruption scenarios.
5. **Cybersecurity:** minimize data collection, bound expensive API operations, validate all inputs, secure dependencies/secrets, and document a Take9-aligned threat model.

1. The central research question

The normal version of a location-allocation problem asks:

> Where should a fixed number of resources be placed to maximize coverage or reduce modeled travel time?

GridAid goes further:

> **Where should those resources be placed so that the network remains effective across several plausible disruptions, not only under normal conditions?**

This creates a measurable difference between two kinds of optimization.

Static optimization

The algorithm optimizes one normal road-and-demand state.

```
Normal roads
Normal demand
All resources available
        ↓
Static optimizer
        ↓
Best normal-condition configuration
```

Resilience optimization

The algorithm chooses one configuration that is evaluated across multiple scenarios.

```
Normal
Winter slowdown
Major corridor closure
Resource unavailable
Temporary event demand
Combined stress
        ↓
Shared site-selection decision
        ↓
Average + worst-case objective
        ↓
Resilient configuration
```

A static optimum may achieve the strongest score on a perfect day but degrade sharply during a disruption. A resilient configuration can accept a small normal-condition tradeoff in exchange for stronger average and worst-case performance.

That comparison is one of the project's most important competition visuals.

2. Why Iowa is a strong launch environment

Iowa is not simply the location of the submission. It provides the technical conditions that make the resilience problem meaningful:

- large rural road networks;
- geographically dispersed communities;
- winter road conditions;
- construction and closure events;
- public transportation data;
- active experimentation with technology-supported rural first response.

Iowa United First Aid is particularly relevant to the project's motivation. State materials describe an effort to use technology and trained community volunteers during the period before traditional ambulance arrival in participating

rural areas. The project does not attempt to replace that dispatch system. Instead, it examines a prior planning question: how well are limited resources geographically positioned, and how resilient is that positioning to disruption?

Iowa DOT also publishes 511 feeds containing incidents, construction, special events, cameras, and winter road conditions. Iowa DOT's public GIS/data systems provide road, traffic, and historical crash information that can support both disruption scenarios and the machine-learning subsystem.

This gives GridAid an Iowa-rooted case study while preserving national applicability.

3. What makes the project competition-worthy

The 2026 Congressional App Challenge rules identify idea quality/originality, implementation/UX/design, and demonstrated coding/programming skill as core judging considerations. The project should make those qualities visible rather than forcing judges to infer them.

Dimension	What GridAid demonstrates
Idea quality	Resilience-aware planning rather than only static placement
Iowa relevance	Rural geography, winter disruption, Iowa DOT data, first-response motivation
Programming depth	Graph routing, matrices, integer optimization, ML, simulation, secure API design
UX	One visual workflow: baseline → optimize → disrupt → resilience optimize
Data science	Reproducible public-data pipeline, ML evaluation, sensitivity analysis
Validation	NetworkX comparison, brute-force small cases, held-out ML metrics, Monte Carlo
Cybersecurity	Data minimization, input bounds, rate limiting, secure headers, dependency/secret scanning

The app should not compete by having the largest feature count. Its strength should come from a small number of deeply implemented systems.

4. Product experience

The main interface should behave like a professional planning tool without overwhelming the user.

Main workflow

```
BASELINE
   ↓
STATIC OPTIMIZE
   ↓
STRESS TEST
   ↓
RESILIENCE OPTIMIZE
   ↓
COMPARE + VALIDATE
```

Main controls

- study area;
- resource count;
- optimization objective;
- coverage threshold;
- demand model;
- disruption scenario set;
- Static Optimize;
- Resilience Optimize.

Demand selector

```
Demand Model
[ Population ] [ ML Incident Risk ] [ Blended ]
```

Results

The result panel should compare:

```
Baseline
Static optimum
Resilient optimum
```

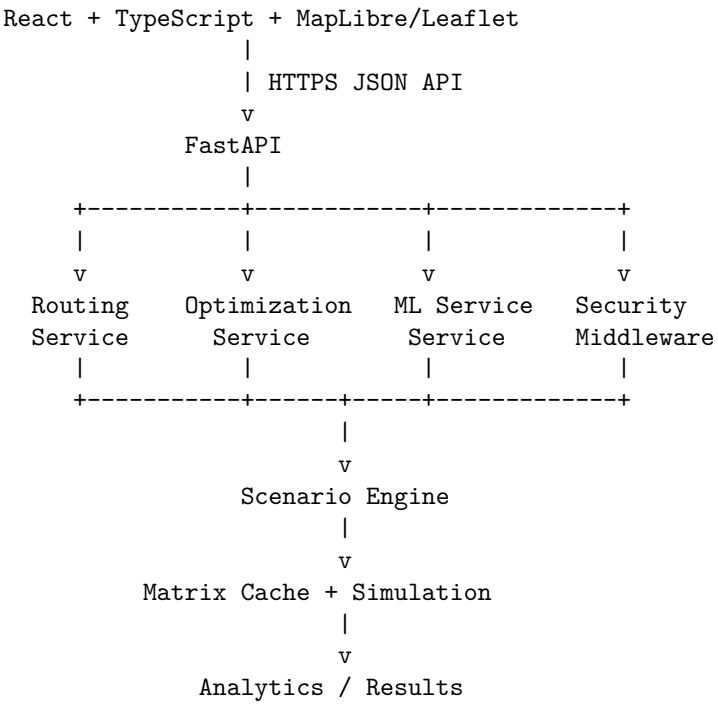
Across:

- normal coverage;
- average scenario coverage;
- worst-case coverage;
- weighted mean modeled travel time;
- p95 modeled travel time;
- percentage within threshold;
- solver runtime.

Map layers

- road network;
- population demand;
- ML incident-risk demand;
- candidate sites;
- selected resources;
- disrupted roads;
- coverage heatmap;
- improvement heatmap.

5. System architecture



The architecture intentionally separates routing, optimization, ML, simulation, and security concerns. That separation makes the project easier to test and prevents the frontend from becoming a collection of hidden calculations.

6. Geospatial data pipeline

GridAid's technical work begins before optimization.

```
OpenStreetMap roads
U.S. Census population
Iowa DOT traffic/crash data
Iowa 511 scenarios
        |
        v
Schema validation
Coordinate-system handling
Geometry cleaning
Spatial joins / snapping
Feature engineering
        |
        +--> road graph
        +--> population demand
        +--> ML feature table
        +--> scenario definitions
        +--> candidate sites
```

Important professional practices:

- every source has a documented retrieval date;
- processed data can be rebuilt from scripts;
- coordinate reference systems are explicit;
- point-to-road snapping distance is stored and audited;
- synthetic resources are labeled synthetic;
- scenario assumptions are labeled assumptions;
- cached public feeds keep source timestamps.

7. Graph routing

The road network is represented as a weighted directed graph `G = (V, E)`.

- `V`: intersections and road endpoints;
- `E`: traversable road segments;
- edge weight: modeled travel time.

For a road edge:

```
base_time = length / modeled_speed
```

A scenario can modify that weight:

```
scenario_time = base_time × condition_multiplier
```

A complete closure removes the edge from traversal.

Algorithms

GridAid should implement and understand Dijkstra's algorithm, then validate results against NetworkX. A* can be used for targeted point-to-point routing if it improves performance, but the core many-to-many problem benefits from Dijkstra/multi-source methods and matrix precomputation.

Why the matrix matters

Optimization should not rerun thousands of graph searches inside every solver decision.

The preprocessing stage creates:

```
T[candidate_site][demand_point] = modeled travel time
```

For 150 candidates and 2,000 demand points, that is 300,000 numeric values—a practical matrix for fast optimization.

Scenario-specific matrices can be cached:

```
matrix_normal.npy
matrix_winter.npy
matrix_closure.npy
matrix_event.npy
```

This design is one of the main reasons the final app can feel fast.

8. Static facility-location optimization

Static optimization establishes the benchmark.

Maximum coverage

Choose exactly `p` candidate sites to maximize weighted demand reachable within a chosen threshold.

Conceptually:

```
maximize covered weighted demand
subject to exactly p selected resources
```

p-median

Choose exactly `p` sites to minimize population/demand-weighted modeled travel time.

p-center / worst-served analysis

A later experiment can minimize the maximum modeled travel time to any demand point.

Validation

On small problems, brute-force every possible configuration and compare it with OR-Tools. For example:

```
8 candidates choose 3 = 56 configurations
```

If the solver does not match brute force, the formulation is wrong.

9. Resilience-aware optimization

This is the signature analytical system.

Let a configuration `C` be evaluated under scenarios `S`.

For each scenario:

```
coverage_s(C)
mean_time_s(C)
p95_time_s(C)
```

A competition-ready resilience score can combine average and worst-case performance:

```
resilience_score(C)
 = alpha × average_s(coverage_s(C))
 + beta × minimum_s(coverage_s(C))
 - gamma × relocation_penalty(C)
```

The weights must be documented and sensitivity-tested.

A stronger solver formulation shares the same site-selection variables across all scenarios while coverage/assignment variables vary by scenario. The optimizer therefore finds **one configuration** that works across the scenario portfolio rather than finding a different optimum for each scenario.

Competition comparison

The final chart should compare:

```
Synthetic baseline
Static normal-condition optimum
Resilience-aware optimum
```

under the same scenarios.

The expected pattern is not that the resilient configuration wins every cell. Instead, it should demonstrate the tradeoff between peak normal-condition performance and stability under disruption.

10. Machine learning subsystem

Machine learning is part of the application, but it must solve a documented problem and have independent validation.

ML task

The recommended subsystem estimates **relative transportation incident risk** in Iowa using historical public crash and roadway/traffic features.

The result becomes an alternative demand layer.

It does **not** claim to predict every medical emergency or actual emergency-response need.

Possible unit of prediction

- road segment × month; or
- spatial cell × month.

Potential features

- AADT / traffic volume;
- truck AADT;
- road class;
- speed limit;
- segment length;
- intersection density;
- urban/rural indicator;
- historical lagged crash frequency;
- season/month;
- surface or weather categories where correctly aligned;
- work-zone indicator.

Target options

- future crash count;
- severity-weighted future crash count;
- binary/ordinal high-risk class.

Models

The project should compare at least:

1. naive historical baseline;
2. simple statistical/linear baseline;
3. tree ensemble such as Random Forest or HistGradientBoosting.

The final model is the one that performs credibly and is explainable—not necessarily the most complex.

Evaluation

Prefer a time-based or grouped split rather than a random row split that leaks geography/time.

Regression metrics:

- MAE;
- RMSE;
- rank correlation if risk ordering matters.

Classification metrics:

- precision/recall;
- PR-AUC;

- ROC-AUC;
- calibration.

Model card

The final repository should include:

- intended use;
- out-of-scope use;
- data dates;
- feature list;
- target definition;
- split method;
- metrics;
- limitations;
- model artifact/version;
- privacy/security notes.

11. How ML affects optimization

GridAid should let users compare demand assumptions.

Population

```
w_i = population_i
```

ML incident risk

```
w_i = normalized predicted risk_i
```

Blended

```
w_i = lambda × normalized_population_i
    + (1-lambda) × normalized_risk_i
```

The result is not "AI chooses the resource locations."

Instead:

```
ML estimates one demand signal
Optimization uses that signal mathematically
Simulation evaluates the resulting placement
```

That distinction makes the technical architecture more defensible.

12. Scenario engine

The scenario system should be deterministic and explainable.

Required scenarios

Normal

Base road network and demand.

Winter slowdown

Apply documented multipliers to affected roads or a cached Iowa 511 winter-condition scenario.

Road closure

Disable selected edges and recompute affected routes.

Resource failure

One or more selected resources become unavailable.

Event demand

Add temporary demand at a fairground, stadium, festival, school, or other event location.

Combined stress

Combine two conditions for a stronger resilience test.

Provenance

Every scenario should identify its source:

```
Live public feed
Cached public snapshot
Expert-informed assumption
Synthetic demo assumption
```

That provenance should be visible in the UI.

13. Monte Carlo validation

The simulation system evaluates configurations across many synthetic incidents rather than one convenient example.

A demand layer becomes a probability distribution:

```
probability_i = weight_i / SUM(weights)
```

Sample 10,000+ incidents with a fixed official-demo seed.

For each configuration, report:

- mean;
- median;
- p90;
- p95;
- percent within threshold;
- worst scenario;
- average scenario coverage;
- minimum scenario coverage.

The final app should display distributions rather than only a single average.

14. Robustness and sensitivity

The project should test whether its recommendation is stable.

Vary:

- resource count;
- coverage threshold;
- winter multiplier;
- disrupted corridor;
- resource availability;
- population-vs-ML demand blend;
- average-vs-worst-case resilience weight.

Site stability

For each site:

```
stability = scenarios_selected / scenarios_tested
```

A site that appears across most reasonable assumptions is more robust than a location selected only by one narrow model.

15. Explainability

A professional planning tool should show why a site was selected.

Placement explanation

- unique demand covered;
- marginal improvement;
- redundancy with neighboring sites;
- modeled access to weak areas.

Resilience explanation

- number of scenarios in which the site remains selected;
- normal contribution;
- worst-case contribution;

- scenario that benefits most from the site.

ML explanation

- model version;
- predicted relative risk;
- feature-importance summary;
- historical data period;
- limitation statement.

Feature importance must not be described as proof of causation.

16. Cybersecurity and the Take9 pledge

The Take9 pledge should be reflected in architecture rather than added only to the submission form.

The public GridAid demo should intentionally collect almost no user data.

Data minimization

Do not collect:

- names;
- emails;
- passwords;
- medical information;
- patient information;
- live 911 records;
- private volunteer home addresses;
- precise user location;
- arbitrary uploaded files.

The app does not need that information to demonstrate its purpose.

Threat model

Assets to protect:

- result integrity;
- API availability;
- model/data artifact integrity;
- deployment secrets;
- trust in public-data timestamps/provenance.

Main threats:

1. expensive optimization requests used for denial of service;
2. malformed inputs designed to crash or exhaust the backend;
3. vulnerable dependencies;
4. secrets accidentally committed to Git or frontend bundles;
5. malformed/compromised third-party feed data;
6. XSS or insecure browser configuration;
7. overly broad CORS;
8. arbitrary model/pickle loading;
9. stale data presented as current;
10. future feature creep that begins collecting sensitive data.

17. Security controls

API validation

Use Pydantic schemas and enforce:

- enum/allowlist fields;
- resource count bounds;
- threshold bounds;
- scenario count bounds;
- maximum simulation trials;
- maximum request size;
- rejected unknown identifiers.

Resource-exhaustion protection

- rate-limit optimization and simulation endpoints;
- server-side timeouts;
- cap candidates/resources/trials;
- cache standard scenarios;
- reject oversized jobs.

Web hardening

- HTTPS;
- HSTS when hosting permits;
- Content-Security-Policy;
- `X-Content-Type-Options: nosniff`;
- restricted CORS allowlist;
- no debug stack traces;
- avoid unsafe HTML rendering.

Secrets

- environment variables/hosting secret store;
- no secrets in repository;
- no private secrets in frontend code;
- `.env` in `.gitignore`;
- secret scan before release.

Dependencies

Run and document:

```
pip-audit
npm audit
Bandit
Gitleaks or equivalent
```

Optionally run an OWASP ZAP baseline scan against the deployed app.

External data

Treat public feeds as untrusted input:

- official host allowlist;
- schema validation;
- timeouts;
- size limits;
- cached fallback;
- source timestamp.

ML artifact security

Never load user-supplied pickle/joblib models. Deploy only a known offline-trained artifact with a fixed feature schema and pinned dependency version.

Why no accounts?

For the CAC version, accounts are unnecessary. Avoiding login/password storage is a security improvement, not a missing feature.

18. Take9 statement inside the app

The Methodology / Security page can state:

> **GridAid follows the Take9 principle by treating security as a design requirement: the public demo minimizes collected data, validates and bounds all planning inputs, rate-limits expensive operations, protects secrets and dependencies, and clearly labels the source and freshness of external data.**

This needs only a short mention in the 3-minute video. The detailed controls live in the product documentation and repository.

19. Backend API

Recommended endpoints:

```
GET  /api/health
GET  /api/study-area
GET  /api/baseline
GET  /api/demand-layers
POST /api/optimize/static
POST /api/optimize/resilient
POST /api/scenarios/evaluate
POST /api/simulate
GET  /api/ml/model-card
GET  /api/ml/risk-layer
GET  /api/runs/{id}/explanation
```

Example request:

```
{
  "resource_count": 10,
  "threshold_minutes": 10,
  "demand_model": "blended",
  "scenario_ids": [
    "normal",
    "winter",
    "closure_demo",
    "resource_failure"
  ],
  "average_weight": 0.6,
  "worst_case_weight": 0.4
}
```

Every expensive endpoint should validate bounds before work begins.

20. Performance plan

The app should feel nearly instant in the final demo.

Precompute

- processed road graph;
- demand points;
- candidate sites;
- normal scenario matrix;
- prepared winter matrix;
- prepared closure matrix;
- ML risk layer.

Cache key

```
study area
+ scenario definition
+ candidate set
+ demand model
+ model version
```

Runtime targets

These are targets, not claims until measured:

- app usable within a few seconds of load;
- standard static optimization near or under 1–2 seconds;
- resilience optimization fast enough for live demonstration;
- 10,000-trial simulation near-instant or a few seconds maximum;
- layer toggles immediate.

The final submission should report real measured runtimes.

21. Validation matrix

System	Validation method
Shortest paths	Hand-calculated toy graph + NetworkX comparison
Static optimizer	Brute-force enumeration on small problem
Resilient optimizer	Toy network where robust solution should differ
Monte Carlo	Fixed seed + reference NumPy statistics
ML	Held-out time/group-aware test set + baseline model
Geospatial	Snapping-distance audit + visual inspection
Security	Input tests, rate-limit test, audits, headers, secret scan
UX	5–10 users attempt workflow without instruction

22. ML experiment plan

Experiment A — baseline

Compare simple historical/constant prediction with tree models.

Experiment B — temporal/generalization test

Train on earlier data and evaluate on held-out later data.

Experiment C — demand-layer comparison

Run the same resource count under:

```
Population
ML incident risk
Blended
```

Measure how recommended sites and coverage metrics change.

Experiment D — model ablation

If time permits, remove one feature family at a time to see whether the model is actually using meaningful roadway/traffic context.

23. Optimization experiment plan

Experiment A — algorithm comparison

```
Random placement
Greedy placement
OR-Tools static optimum/best solution
```

Report objective and runtime.

Experiment B — static vs resilient

Compare the static optimum and resilient optimum across all core scenarios.

Experiment C — resource-count curve

Run several resource counts and plot diminishing returns.

Experiment D — scenario sensitivity

Vary the disruption assumptions and measure site-selection stability.

24. Expert outreach

The strongest feedback comes from people who understand emergency planning, GIS, dispatch data, or Iowa first-response systems.

Initial outreach targets:

- Scott County Emergency Management;
- Scott Emergency Communications Center / GIS-data staff;
- Iowa United First Aid / Center for Rural Revitalization;
- a locally referred EMS/fire/public-safety professional;
- Iowa DOT data/GIS staff only when technical data questions require it.

Questions should focus on modeling assumptions, not endorsement:

- Which geographic factors matter when planning coverage?
- How do road closures or winter conditions change planning?
- Is average modeled access useful, or are worst-served areas more important?
- What would make a student model unrealistic?
- Is population a reasonable baseline demand proxy?
- What non-sensitive public data could strengthen the model?
- Does the resilience comparison communicate something useful?

The submission can state that professionals were consulted only if the conversations actually happen.

25. Four-week execution schedule

September 25–29

- repository and security baseline;
- synthetic graph;
- Dijkstra;
- travel matrix;
- greedy optimizer;
- OR-Tools static model;
- brute-force validation;
- outreach begins.

September 30–October 4

- scenario engine;
- first robust objective;
- real Iowa road graph;
- Census demand;
- candidate sites;
- normal matrix.

October 5–10

- Iowa DOT ML dataset;
- feature engineering;
- baseline model;
- tree model;
- held-out evaluation;
- model card;
- incident-risk layer;
- prepared disruption matrices.

October 8–14

- FastAPI;
- React/TypeScript map;
- static optimization UI;
- scenario UI;
- resilience optimization UI;
- ML demand toggle;
- explainability.

October 13–18

- Monte Carlo;
- robustness experiments;
- performance benchmarks;
- security hardening/scans;
- expert feedback;
- user testing.

October 18–20

- documentation;
- methodology/security pages;
- final UX;
- model/data provenance;
- final naming search.

Feature freeze: October 20.

October 21–24

- final tests;
- CAC written answers;
- record demo video;
- rehearse ten complete runs;
- backup deployment.

October 25

Submit.

26. What not to build

To keep the project professional and finishable, do not add:

- live 911 integration;
- medical diagnosis or treatment instructions;
- user accounts/passwords;
- social feeds;
- chatbots;
- arbitrary file uploads;
- nationwide support before the Iowa study works;
- unrelated computer vision;
- push notifications;
- mobile-native rebuild;
- deep learning solely for prestige;
- unsupported “lives saved” numbers.

Professional quality comes from correctness, validation, security, speed, provenance, and UX—not feature count.

27. Three-minute CAC video

Target: approximately **2:50**, leaving time margin.

0:00–0:13 — Hook

> "I'm [name], and I built [final project name]. A resource network that works on a normal day can fail when a road closes, winter weather slows travel, a resource becomes unavailable, or demand shifts."

0:13–0:28 — Purpose and audience

> "[Name] is a planning simulator for emergency-management and community planners. It tests where limited resources can be placed so geographic coverage remains strong across changing conditions."

0:28–0:58 — Static optimization

Show baseline and press **Static Optimize**.

> "I model roads as a weighted graph and calculate network travel costs from candidate sites to demand locations."

0:58–1:28 — Disruption + resilience

Activate the prepared road closure/winter scenario.

Show degraded static configuration.

Press **Resilience Optimize**.

> "Instead of optimizing one perfect day, the resilience model searches for one configuration that performs well across normal, winter, closure, resource-failure and demand-shift scenarios."

1:28–1:50 — Architecture

Show one compact diagram.

> "The backend uses Python and FastAPI, GeoPandas for geospatial processing, graph algorithms for routing, OR-Tools for optimization, scikit-learn for the Iowa incident-risk model, and NumPy for simulation."

1:50–2:12 — Machine learning

Toggle the ML risk layer or display a compact model panel.

> "The ML subsystem uses historical Iowa transportation data to estimate relative transportation-incident risk. I evaluate it on held-out data and use it as an alternative demand layer rather than pretending it predicts every emergency."

2:12–2:31 — Validation

Show:

```
Routes vs NetworkX
Optimizer vs brute force
10,000 simulations
Held-out ML metric
```

2:31–2:41 — Cybersecurity

> "The public demo follows the Take9 pledge by collecting no personal or medical data, validating and rate-limiting API inputs, protecting secrets, scanning dependencies, and treating external data as untrusted."

2:41–2:54 — Close

> "[Name] makes resilience planning measurable: not just where resources work today, but whether they still work when conditions change."

28. What stays out of the video

The video does not need to teach:

- p-median equations;
- Dijkstra pseudocode;
- all model hyperparameters;
- database schema;
- coordinate-system details;
- every security header;
- every expert interview;
- all unit tests;
- source-code organization.

Those belong in the app, repository, methodology, and written answers.

29. Judge-facing technical explanation

A one-minute technical explanation should be memorized:

> "I represent the road network as a weighted graph and snap candidate resource sites and demand points to that graph. I precompute candidate-to-demand travel-time matrices so optimization is fast. The static model uses facility-location optimization to maximize coverage or reduce weighted travel time. The resilience model shares the same selected locations across several disruption scenarios and rewards both average and worst-case performance. Separately, I train an Iowa transportation incident-risk model on historical public data and use its output as another demand layer. Finally, I validate configurations with Monte Carlo simulation and independently test the routing, optimizer and ML model."

30. Cybersecurity explanation for judges

> "I treated cybersecurity as part of the design. The demo does not need accounts or personal data, so I removed those risks. All POST requests use strict typed schemas and bounds, expensive solver endpoints are rate-limited and timed out, CORS is restricted, security headers are enabled, secrets stay outside the repository, dependencies and code are scanned, and external feeds are validated before processing. I also never accept user-supplied model files."

That answer is stronger than saying only "I used HTTPS."

31. Submission-answer drafts

Purpose

GridAid is a resilience planning simulator that uses road-network algorithms, optimization, machine learning and simulation to test where limited emergency resources can be placed so coverage remains effective when roads, demand or resource availability change.

Inspiration

I wanted to build an Iowa-rooted project around a real technical problem rather than a stereotypical theme. Iowa's rural geography, winter road conditions, public transportation data and technology-supported first-response initiatives made me interested in a deeper question: a resource layout can look optimal on a normal day but fail during a disruption. GridAid is designed to measure and optimize that resilience.

Technical challenge

The final answer should use a challenge actually encountered. Strong candidates include:

- generating many-to-many road travel times fast enough for interactive optimization;
- constructing one resource configuration across multiple scenario matrices;
- preventing data leakage in the ML training set;
- correctly invalidating route/matrix caches after a disruption.

Learning

I learned that advanced data science is not just training a model. I had to define honest targets, prevent data leakage, validate graph and optimization algorithms independently, measure sensitivity to assumptions, and design the public API so computational features could not be abused.

Version 2.0

A future version would use professionally validated emergency-demand data, expand to more regions and hazard types, partner with planning professionals, improve robust optimization, and undergo formal security and operational review before any real-world decision use.

32. Release gate

The application is competition-ready only if all of the following are true.

Algorithms

- routing tests pass;
- static optimizer matches brute force on small cases;
- resilient toy case behaves as expected;
- simulation is reproducible.

Data

- Iowa road graph reproducible;
- Census demand documented;
- ML training data documented;
- synthetic resources labeled;
- scenario provenance visible.

ML

- baseline model compared;
- leakage review completed;

- held-out metric recorded;
- model card complete;
- inference schema fixed.

Security

- no personal/medical data collection;
- no arbitrary upload;
- request bounds;
- rate limits;
- timeouts;
- restricted CORS;
- security headers;
- dependency audit;
- Python security scan;
- secret scan;
- no debug mode;
- HTTPS deployment.

Product

- baseline/static/resilient workflow understandable;
- prepared disruption reliable;
- ML layer loads quickly;
- no fake metrics;
- no broken controls;
- methodology/security page complete.

Submission

- final name collision search;
- video under 3 minutes;
- participant/app/purpose/audience/tools included;
- AI/ML use disclosed accurately;
- written answers complete;
- demo succeeds ten consecutive times;
- submit October 25.

33. Source and research notes

2026 Congressional App Challenge Rules
https://www.congressionalappchallenge.us/wp-content/uploads/2026/05/2026-CAC-Rules.pdf

Congressional App Challenge rules / submission information
https://www.congressionalappchallenge.us/students/rules/

Iowa United First Aid
https://opportunityiowa.gov/community/revitalization/center-rural-revitalization/iowa-united-first-aid-grant-program

Iowa DOT 511 data feeds
https://iowadot.gov/travel-tools/iowa-511/511-data-feeds

Iowa DOT open data / GIS
https://data.iowadot.gov/

U.S. Census Data API
https://api.census.gov/data.html

OpenStreetMap attribution/license
https://www.openstreetmap.org/copyright

Take9 cybersecurity public-awareness campaign
https://pausetake9.org/

OWASP API Security Project / API Security Top 10
https://owasp.org/projects/api-security-project

OWASP Application Security Verification Standard
https://owasp.org/www-project-application-security-verification-standard/

34. Final project standard

GridAid should feel professional because the important systems are real:

```
verified graph routing
+ facility-location optimization
+ multi-scenario resilience model
+ evaluated Iowa ML risk layer
+ Monte Carlo validation
+ secure bounded API
+ clear data provenance
+ fast, understandable UX
```

The project does not need unrelated features to appear sophisticated. The competition version should be the smallest complete system that proves all of those capabilities and communicates them clearly in under three minutes.